

```
In [23]: import numpy as np
import matplotlib.pyplot as plt
import h5py
```

```
In [24]: with h5py.File("mnist_traindata (1).hdf5","r") as f:
    X_train = f["xdata"][:]
    y_train = f["ydata"][:]

    with h5py.File("mnist_testdata (1).hdf5","r") as f:
        X_test = f["xdata"][:]
        y_test = f["ydata"][:]

    if len(X_train.shape)==3:
        X_train = X_train.reshape(X_train.shape[0],-1)
        X_test = X_test.reshape(X_test.shape[0],-1)

    if len(y_train.shape)>1:
        y_train = np.argmax(y_train,axis=1)
        y_test = np.argmax(y_test,axis=1)

    N,D = X_train.shape
    K = 10

    print(X_train.shape, X_test.shape)
```

(60000, 784) (10000, 784)

```
In [25]: def softmax(z):
    z = z - np.max(z,axis=1,keepdims=True)
    exp = np.exp(z)
    return exp/np.sum(exp,axis=1,keepdims=True)

    def one_hot(y,K=10):
        Y = np.zeros((y.shape[0],K))
        Y[np.arange(y.shape[0]),y] = 1
        return Y

    def loss(X,Y,W,b):
        scores = X@W.T + b
        probs = softmax(scores)
        return -np.sum(Y*np.log(probs+1e-12))/X.shape[0]

    def accuracy(X,y,W,b):
        scores = X@W.T + b
        probs = softmax(scores)
        pred = np.argmax(probs,axis=1)
        return np.mean(pred==y)
```

```
In [26]: lr = 0.5
epochs = 50

W = np.zeros((K,D))
b = np.zeros(K)

Y_train = one_hot(y_train)
Y_test = one_hot(y_test)
```

```

train_loss=[]
test_loss=[]
train_acc=[]
test_acc=[]

for epoch in range(epochs):

    scores = X_train @ W.T + b
    probs = softmax(scores)

    grad = (probs - Y_train)/N

    grad_W = grad.T @ X_train
    grad_b = np.sum(grad,axis=0)

    W -= lr * grad_W
    b -= lr * grad_b

    train_loss.append(loss(X_train,Y_train,W,b))
    test_loss.append(loss(X_test,Y_test,W,b))

    train_acc.append(accuracy(X_train,y_train,W,b))
    test_acc.append(accuracy(X_test,y_test,W,b))

```

```

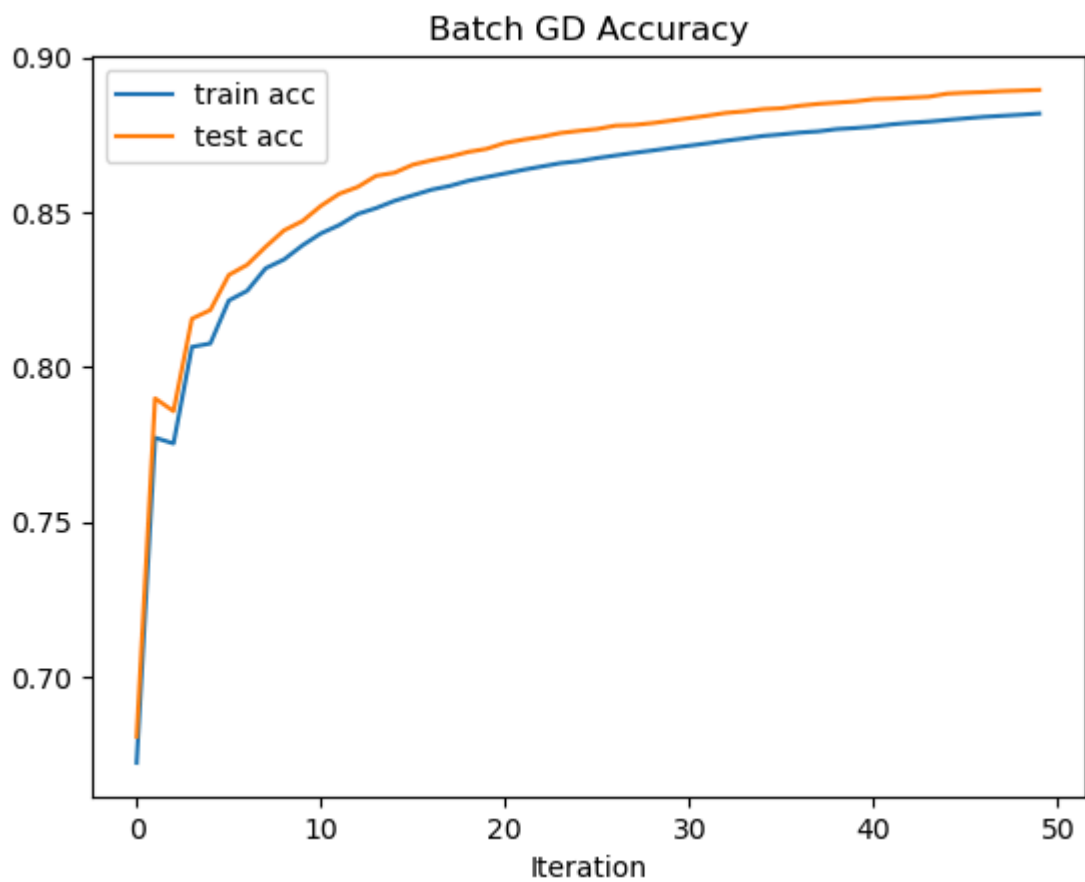
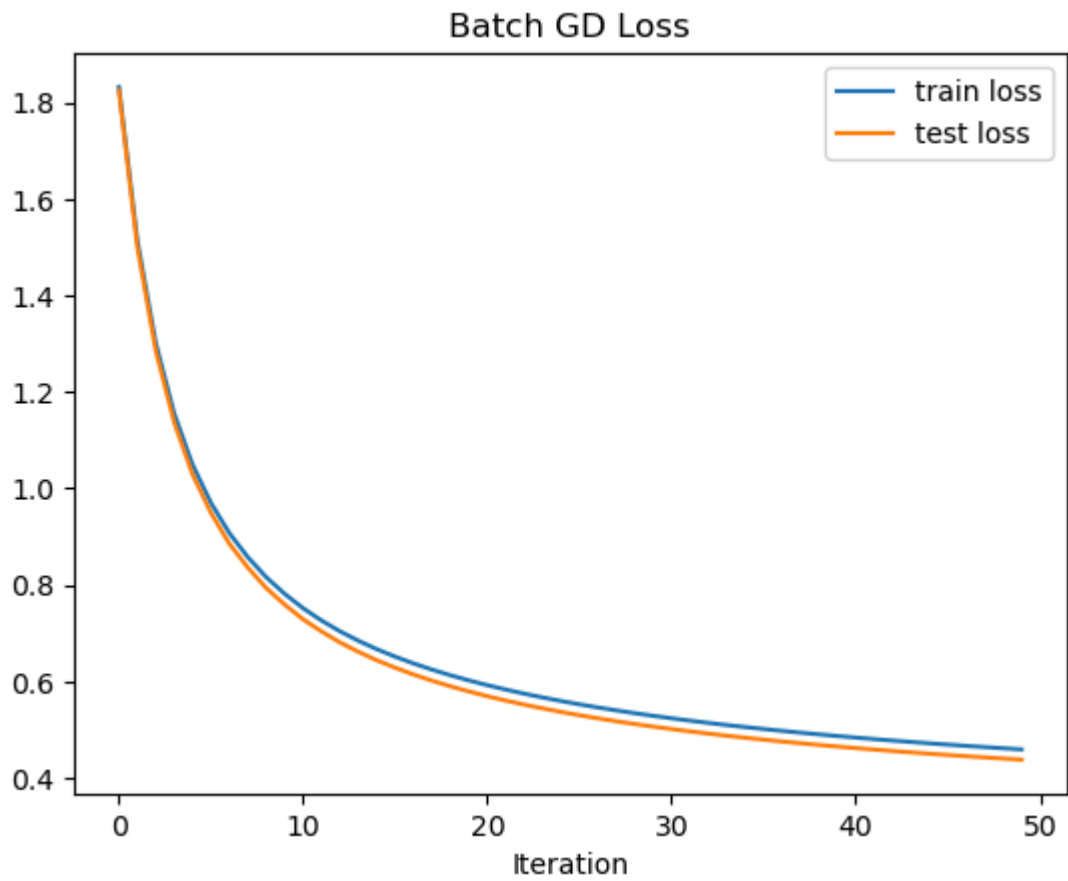
In [27]: iters = np.arange(len(train_loss))

plt.figure()
plt.plot(iters,train_loss,label="train loss")
plt.plot(iters,test_loss,label="test loss")
plt.legend()
plt.xlabel("Iteration")
plt.title("Batch GD Loss")

plt.figure()
plt.plot(iters,train_acc,label="train acc")
plt.plot(iters,test_acc,label="test acc")
plt.legend()
plt.xlabel("Iteration")
plt.title("Batch GD Accuracy")

plt.show()

```



```
In [28]: lr = 0.01
         iterations = 200000

         W_sgd = np.zeros((K,D))
         b_sgd = np.zeros(K)
```

```

sgd_train_loss=[]
sgd_test_loss=[]
sgd_train_acc=[]
sgd_test_acc=[]

for i in range(iterations):

    idx = np.random.randint(N)

    x = X_train[idx:idx+1]
    y = y_train[idx]

    y_vec = np.zeros((1,K))
    y_vec[0,y] = 1

    scores = x @ W_sgd.T + b_sgd
    probs = softmax(scores)

    grad = probs - y_vec

    grad_W = grad.T @ x
    grad_b = grad[0]

    W_sgd -= lr * grad_W
    b_sgd -= lr * grad_b

    if i % 5000 == 0:

        sgd_train_loss.append(loss(X_train,Y_train,W_sgd,b_sgd))
        sgd_test_loss.append(loss(X_test,Y_test,W_sgd,b_sgd))

        sgd_train_acc.append(accuracy(X_train,y_train,W_sgd,b_sgd))
        sgd_test_acc.append(accuracy(X_test,y_test,W_sgd,b_sgd))

```

```

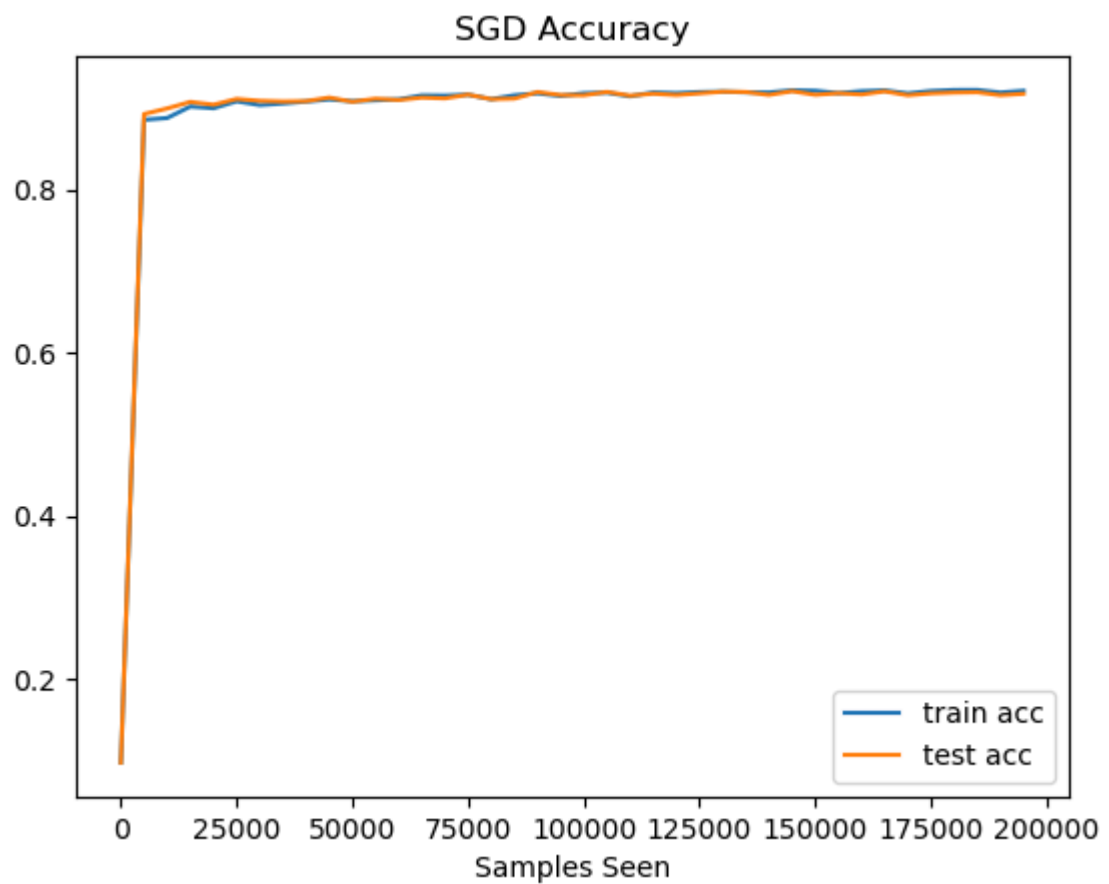
In [29]: samples_sgd = np.arange(len(sgd_train_acc))*5000

plt.figure()
plt.plot(samples_sgd,sgd_train_loss,label="train loss")
plt.plot(samples_sgd,sgd_test_loss,label="test loss")
plt.legend()
plt.xlabel("Samples Seen")
plt.title("SGD Loss")

plt.figure()
plt.plot(samples_sgd,sgd_train_acc,label="train acc")
plt.plot(samples_sgd,sgd_test_acc,label="test acc")
plt.legend()
plt.xlabel("Samples Seen")
plt.title("SGD Accuracy")

plt.show()

```



```
In [30]: lr = 0.1  
batch_size = 100  
epochs = 10  
  
W_mb = np.zeros((K,D))
```

```

b_mb = np.zeros(K)

mb_train_loss=[]
mb_test_loss=[]
mb_train_acc=[]
mb_test_acc=[]

samples_seen = 0

for epoch in range(epochs):

    perm = np.random.permutation(N)

    for i in range(0,N,batch_size):

        batch = perm[i:i+batch_size]

        Xb = X_train[batch]
        Yb = Y_train[batch]

        scores = Xb @ W_mb.T + b_mb
        probs = softmax(scores)

        grad = (probs - Yb)/batch_size

        grad_W = grad.T @ Xb
        grad_b = np.sum(grad,axis=0)

        W_mb -= lr * grad_W
        b_mb -= lr * grad_b

        samples_seen += batch_size

        if samples_seen % 5000 == 0:

            mb_train_loss.append(loss(X_train,Y_train,W_mb,b_mb))
            mb_test_loss.append(loss(X_test,Y_test,W_mb,b_mb))

            mb_train_acc.append(accuracy(X_train,y_train,W_mb,b_mb))
            mb_test_acc.append(accuracy(X_test,y_test,W_mb,b_mb))

```

```

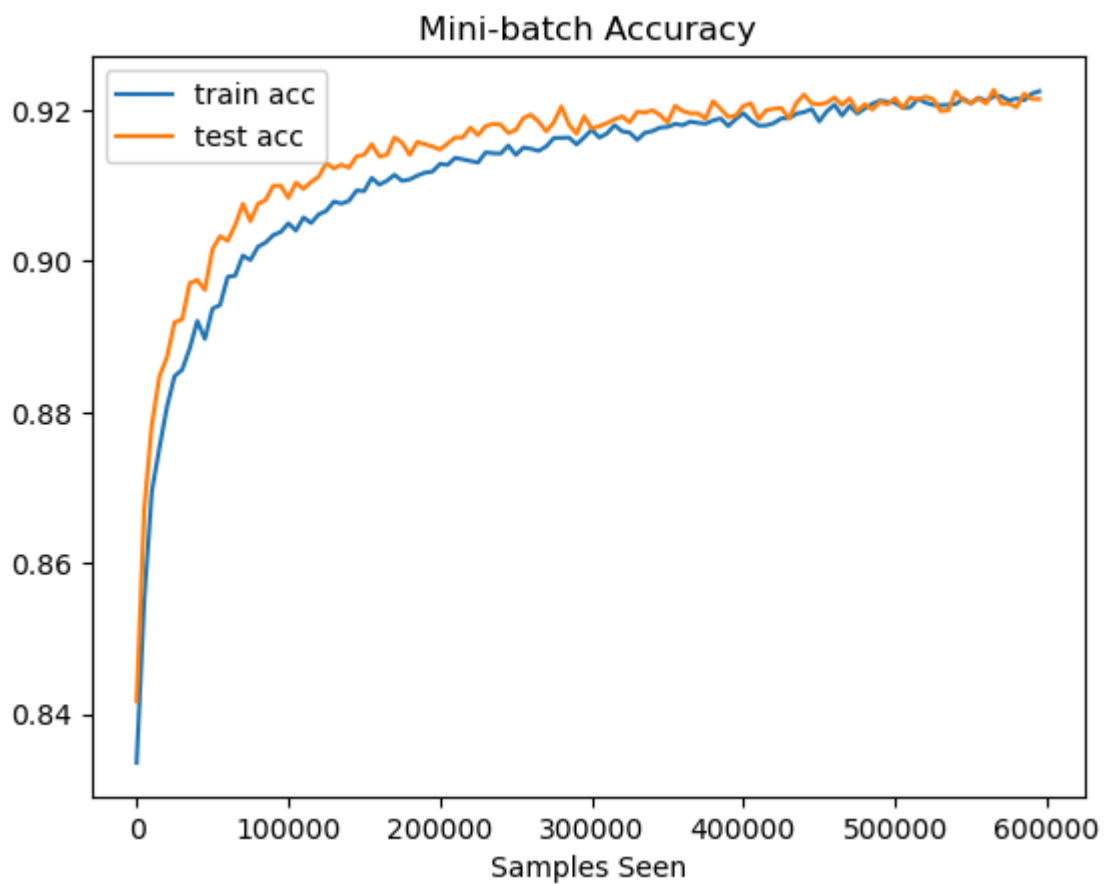
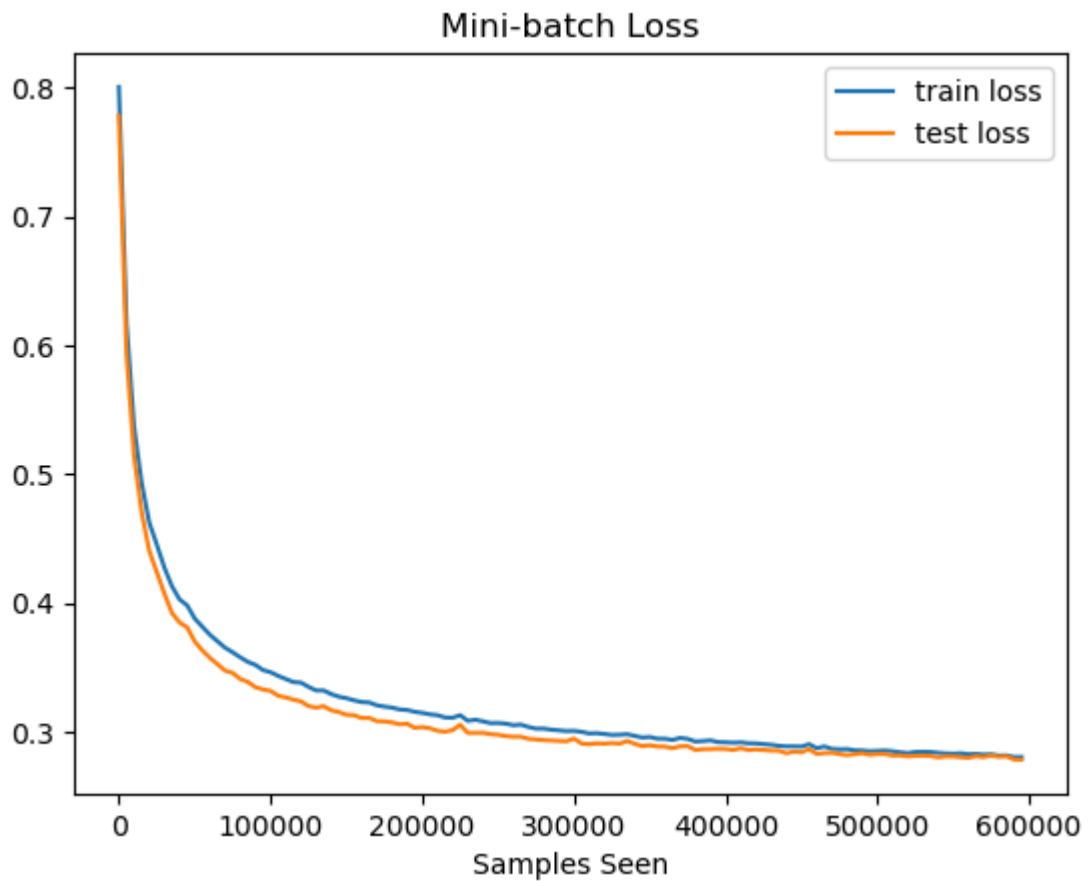
In [31]: samples_mb = np.arange(len(mb_train_acc))*5000

plt.figure()
plt.plot(samples_mb,mb_train_loss,label="train loss")
plt.plot(samples_mb,mb_test_loss,label="test loss")
plt.legend()
plt.xlabel("Samples Seen")
plt.title("Mini-batch Loss")

plt.figure()
plt.plot(samples_mb,mb_train_acc,label="train acc")
plt.plot(samples_mb,mb_test_acc,label="test acc")
plt.legend()
plt.xlabel("Samples Seen")
plt.title("Mini-batch Accuracy")

plt.show()

```



```
In [32]: with h5py.File("weights.hdf5","w") as f:
          f["W"] = W
          f["b"] = b

          print("weights saved")
```

weights saved

1:

From Problem 2 we know that the error vector of the output layer is $\delta = a - y$ where a is the predicted probability vector from the softmax function and y is the one-hot encoded label vector.

Therefore, the gradient of the log-likelihood with respect to the weight vector is $\nabla_w L = (P(Y = l | x, w) - y_l) x$ where $P(Y = l | x, w)$ is the predicted probability of class l .

2:

We implemented batch gradient descent with a learning rate of 0.5, which produced stable convergence during training.

3:

We plotted the training and test log-loss on the same figure to show the learning curve.

We also plotted the training and test accuracy on a separate figure as a function of the iteration number.

4:

After training, the model achieved approximately 0.92 training accuracy and 0.91 test accuracy, indicating that the softmax classifier performs well on the MNIST dataset.

c1:

In stochastic gradient descent (SGD), the model parameters are updated using a single training sample at each step.

We recorded the log-loss and accuracy of the training and test sets every 5000 samples.

SGD converges more noisily compared to batch gradient descent but requires much less computation per update.

2:

Batch gradient descent computes gradients over the entire dataset in each iteration, which results in higher computational cost.

In contrast, SGD updates the parameters using only one sample at a time, making each update cheaper but introducing more variance

3:

Mini-batch SGD uses a batch size of 100, which balances stability and computational efficiency.

Compared to SGD, mini-batch updates reduce gradient variance and lead to more stable training while remaining computationally cheaper than batch gradient descent.

In []: